AWS Infrastructure Report

Provisioning an EC2 Instance and S3 Bucket

using Terraform on AWS

Technical Lab Documentation

Technologies Used

Amazon Web Services (AWS) • Terraform
Docker • GitLab CI/CD • Ubuntu (WSL)

February 2026

0 Contents

1 Introduction

This report documents the full process of provisioning AWS cloud infrastructure using Terraform, deploying a Dockerized application via a GitLab CI/CD pipeline, and hosting the application on an EC2 instance. The workflow covers environment setup, IAM user creation, infrastructure-as-code, container registry integration, and production deployment.

The lab environment uses Windows Subsystem for Linux (WSL) with Ubuntu as the development platform, allowing Linux tooling on a Windows host machine.

2 AWS IAM User Configuration

Before using Terraform with AWS, an IAM user with programmatic access must be created and configured. This section covers searching for IAM, attaching the required policies, and generating access keys.

2.1 Searching for IAM in the AWS Console

Navigate to the AWS Console and search for "IAM" in the search bar. Select the **IAM** service to manage access to AWS resources.

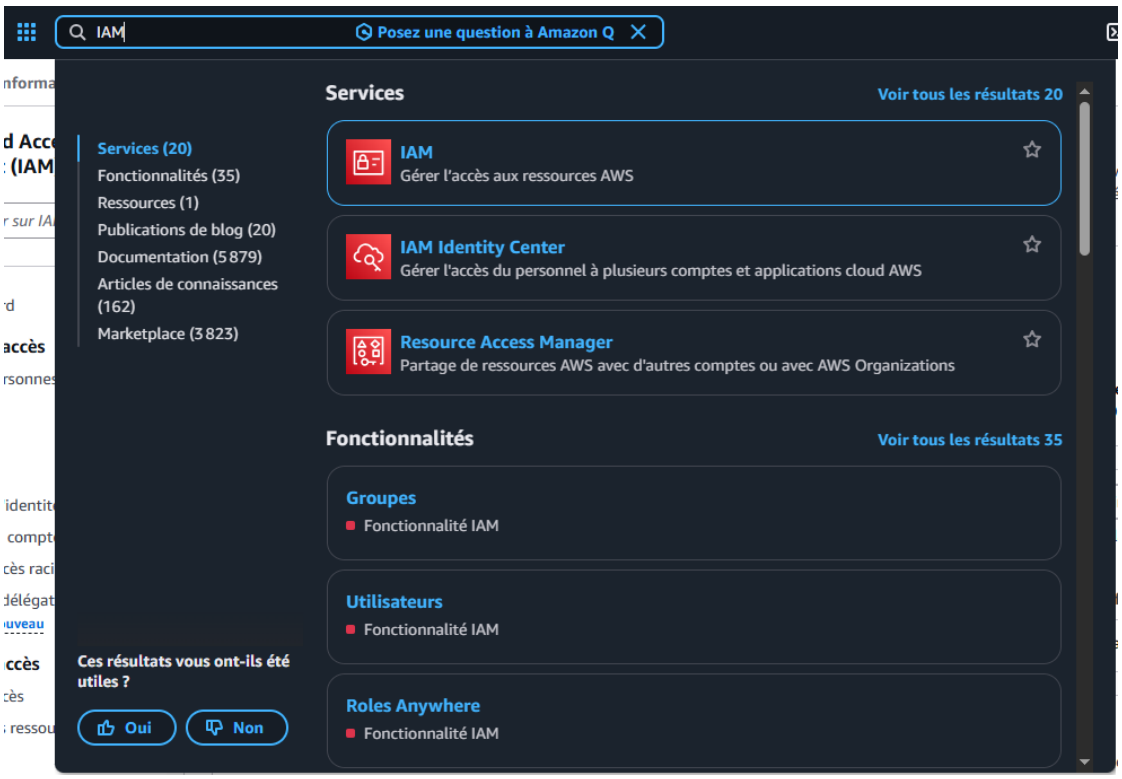

Figure 1: Searching for IAM in the AWS Console — selecting the IAM service

2.2 Attaching S3 Policy to the IAM User

During user creation, attach the appropriate permissions. Search for `s3` in the permissions policies list and select **AmazonS3FullAccess** to grant full access to S3 buckets.

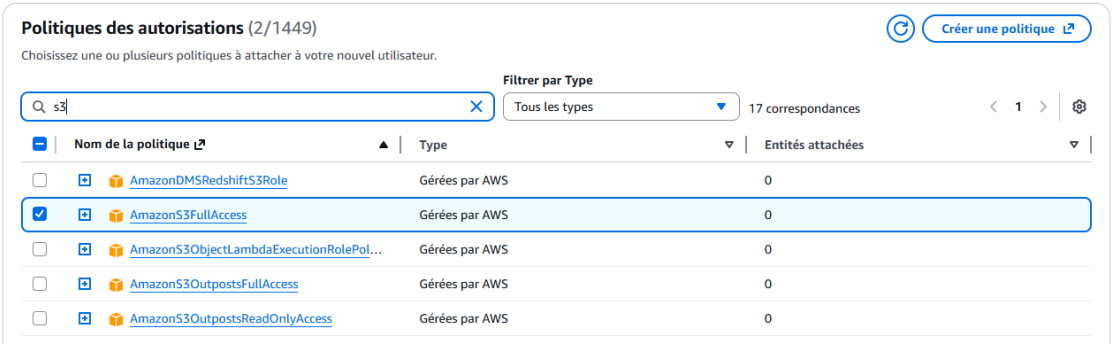

Figure 2: Attaching AmazonS3FullAccess policy to the new IAM user

2.3 Attaching Additional Policies

Additional policies may be required depending on the resources Terraform will manage. Here, **AmazonOmicsFullAccess** is selected alongside the previously added S3 policy.

Figure 3: Selecting additional AWS managed policies for the IAM user

2.4 Attaching AdministratorAccess Policy

For this lab, **AdministratorAccess** is attached to give the Terraform user full control over all AWS services.

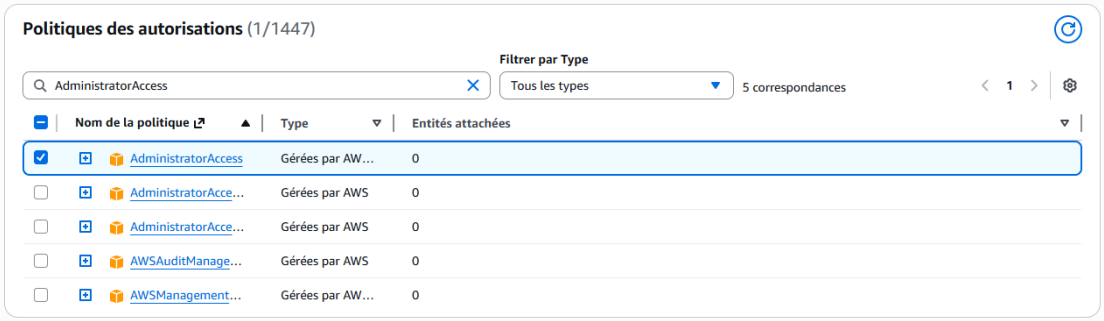

Figure 4: Attaching AdministratorAccess policy — grants full AWS permissions

Security Note: Attaching `AdministratorAccess` is convenient for labs but not recommended in production. Use least-privilege policies in real environments.

2.5 Creating an Access Key for CLI Use

Once the user is created, generate an access key for programmatic access. Select the **Command Line Interface (CLI)** use case, which will allow the AWS CLI and Terraform to authenticate.

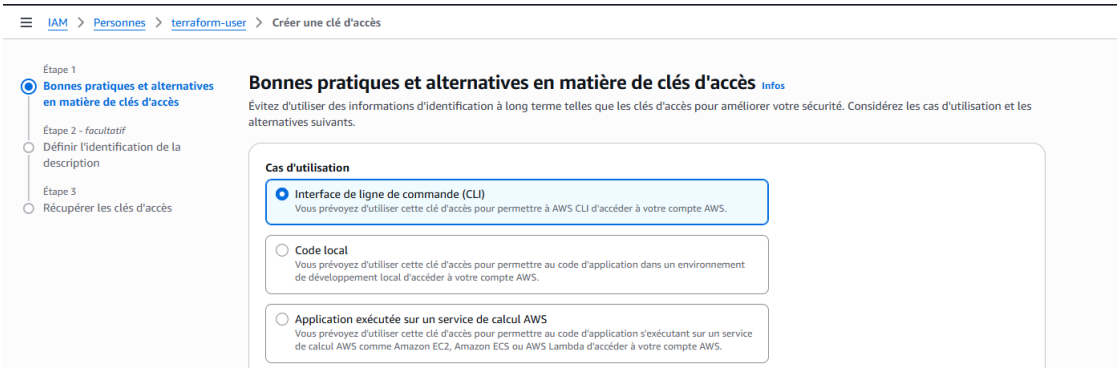

Figure 5: Creating an IAM access key — selecting CLI as the use case

3 Environment Setup

3.1 Installing WSL and Ubuntu

Windows Subsystem for Linux (WSL) must be enabled and Ubuntu installed from the Microsoft Store before proceeding with any tool installation.

3.2 Installing Terraform

Update system and install dependencies:

```
sudo apt update
sudo apt install -y gnupg software-properties-common
```

Add HashiCorp GPG key:

```
wget -O- https://apt.releases.hashicorp.com/gpg | \
gpg --dearmor | \
sudo tee /usr/share/keyrings/hashicorp-archive-keyring.gpg
```

Add repository and install:

```
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-
   keyring.gpg] \
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | \
sudo tee /etc/apt/sources.list.d/hashicorp.list

sudo apt update
sudo apt install terraform
terraform -version
```

3.3 Installing AWS CLI v2

```
sudo apt install unzip curl -y
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip"
   -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
aws --version
```

3.4 Configuring the AWS CLI

Run `aws configure` and enter the IAM credentials obtained in Section 2:

```
aws configure
# Access Key ID:     <from IAM user>
# Secret Access Key: <from IAM user>
# Default Region:    us-east-1
# Output Format:     json
```

4 Creating an EC2 Key Pair

SSH access to the EC2 instance requires a key pair. Navigate to the EC2 service in the AWS Console.

4.1 Navigating to EC2

Search for "ec2" in the AWS Console search bar and select the **EC2** service.

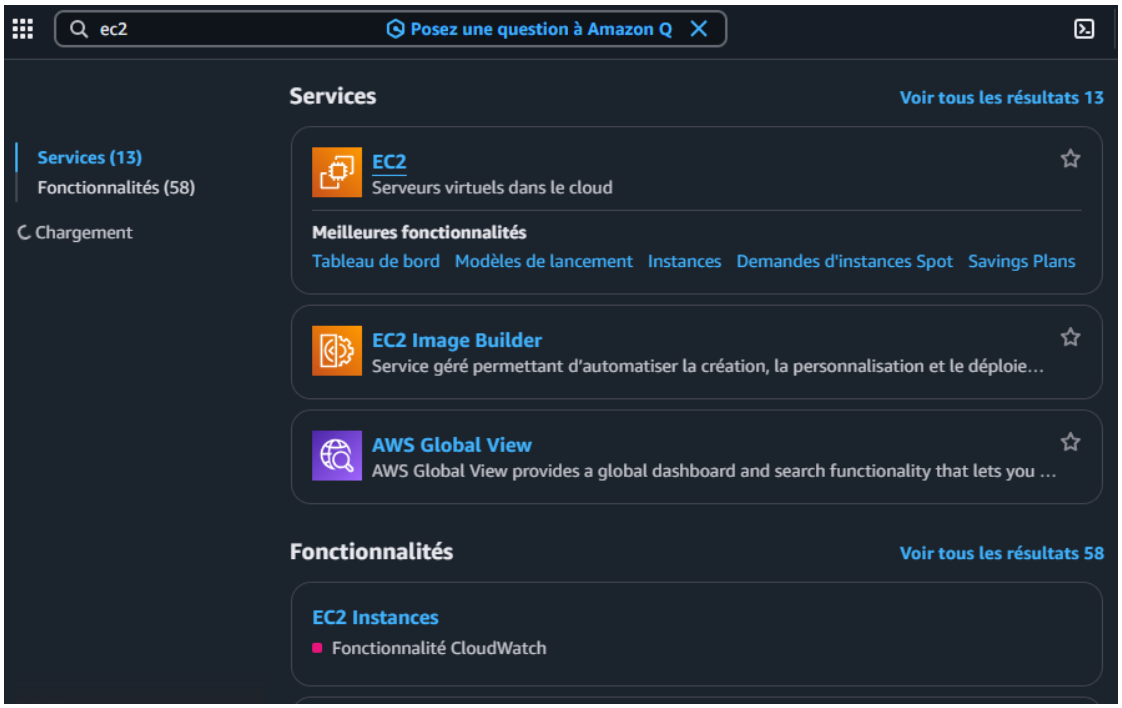

Figure 6: Searching for EC2 in the AWS Console

4.2 Creating the Key Pair

Go to **Network & Security** → **Key Pairs** and click **Create a key pair**. The page initially shows no existing key pairs.

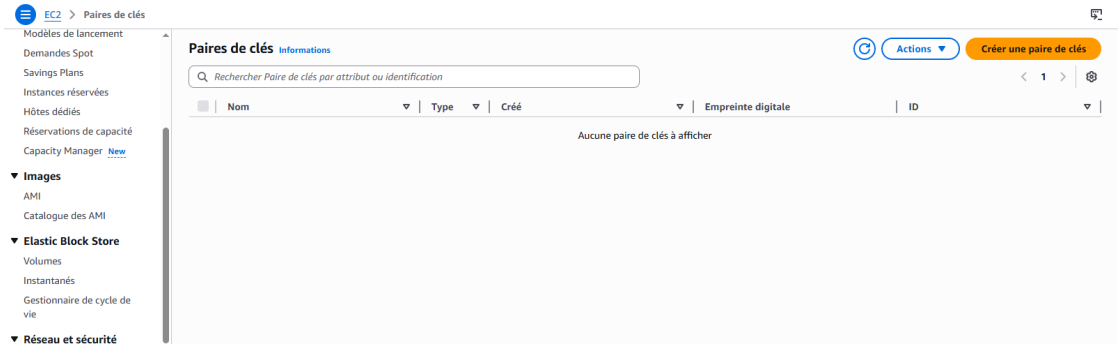

Figure 7: EC2 Key Pairs page — ready to create a new key pair

After creating the key pair named `terraformkeyinstance`, a success confirmation is displayed and the `.pem` file is automatically downloaded.

Figure 8: Key pair created successfully

5 Terraform Infrastructure as Code

5.1 Project Structure

```
mkdir terraform-lab
cd terraform-lab
nano main.tf
```

5.2 Main Terraform Configuration (`main.tf`)

```
# ----------------------------
# AWS Provider Configuration
# ----------------------------
provider "aws" {
  region = "us-east-1"
}

# ----------------------------
# Create S3 Bucket
# ----------------------------
resource "aws_s3_bucket" "my_bucket" {
  bucket = "asma-terraform-bucket-2026-unique"
  tags = {
    Name        = "TerraformBucket"
    Environment = "Dev"
```

```
  }
}

# ----------------------------
# Create Security Group (Allow SSH)
# ----------------------------
resource "aws_security_group" "allow_ssh" {
  name        = "allow_ssh_terraform"
  description = "Allow SSH inbound traffic"
  ingress {
    description = "SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

# ----------------------------
# Create EC2 Instance
# ----------------------------
resource "aws_instance" "my_instance" {
  ami                    = "ami-0f3caa1cf4417e51b"
  instance_type          = "t3.micro"
  key_name               = "terraformkeyinstance"
  vpc_security_group_ids = [aws_security_group.allow_ssh.id]
  tags = {
    Name = "TerraformEC2"
  }
}

# ----------------------------
# Output Public IP
# ----------------------------
output "instance_public_ip" {
  value = aws_instance.my_instance.public_ip
}
```

5.3 Deploying the Infrastructure

```
terraform init    # Initialize providers and modules
terraform plan    # Preview changes before applying
terraform apply   # Provision the infrastructure
```

5.4 Terraform Apply — Result

After running `terraform apply` and confirming with `yes`, Terraform destroys any outdated instance and creates a fresh EC2 instance. The output shows the public IP of the new instance.

```
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_instance.my_instance: Destroying... [id=i-02bba0427a8fbde96]
aws_instance.my_instance: Still destroying... [id=i-02bba0427a8fbde96, 00m10s elapsed]
aws_instance.my_instance: Still destroying... [id=i-02bba0427a8fbde96, 00m20s elapsed]
aws_instance.my_instance: Still destroying... [id=i-02bba0427a8fbde96, 00m30s elapsed]
aws_instance.my_instance: Destruction complete after 31s
aws_instance.my_instance: Creating...
aws_instance.my_instance: Still creating... [00m10s elapsed]
aws_instance.my_instance: Creation complete after 15s [id=i-030358b538d64b939]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

Outputs:

instance_public_ip = "54.196.243.75"
ubuntu@DESKTOP-DJV6BEK:~/terraform-lab$
```

Figure 9: Terraform apply output — EC2 instance created with public IP `54.196.243.75`

5.5 EC2 Instance Visible in AWS Console

The newly provisioned **TerraformEC2** instance (`t3.micro`) appears in the EC2 Instances dashboard with status *En cours d'initialisation* (initializing).

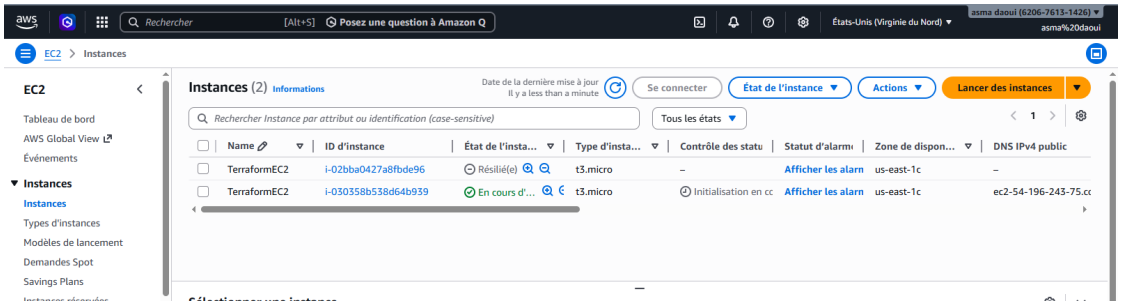

Figure 10: EC2 instances in the AWS Console — TerraformEC2 instance running in us-east-1c

6 GitLab CI/CD Pipeline — Docker Build and Deploy

6.1 Problem: No Docker Image in Registry

Before the pipeline was correctly configured, the GitLab Container Registry showed no images for the `note-app` project.

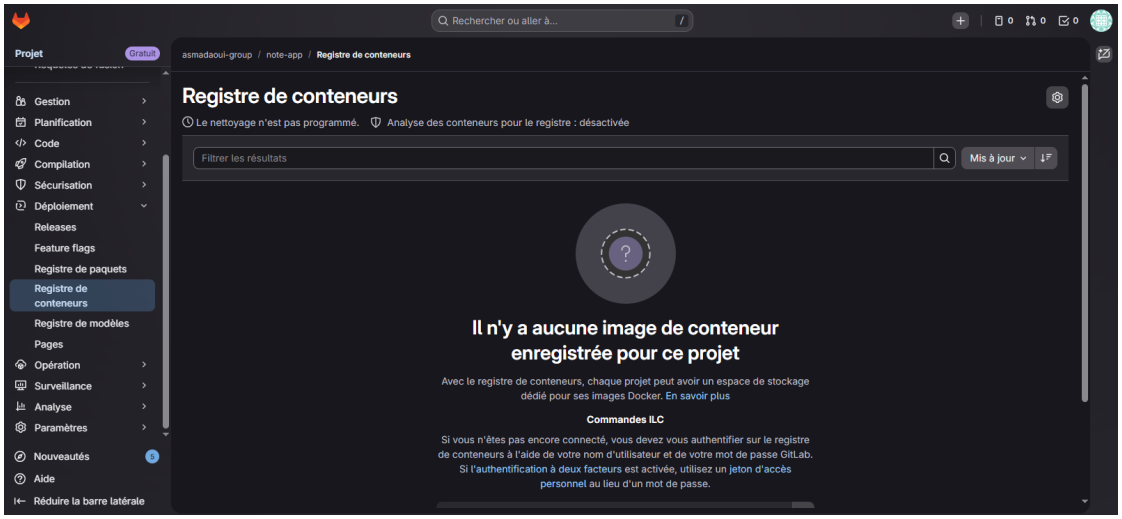

Figure 11: GitLab Container Registry — empty, no Docker image registered yet

6.2 Pipeline Configuration

The `.gitlab-ci.yml` was updated to include a **package** stage that builds and pushes the Docker image, and a **deploy** stage that pulls and runs it.

Package Stage:

```
package-docker:
  stage: package
  image: docker:latest
  services:
    - docker:dind
  variables:
    DOCKER_TLS_CERTDIR: ""
  script:
    - echo "$CI_JOB_TOKEN" | docker login -u "$CI_REGISTRY_USER"
        --password-stdin $CI_REGISTRY
    - docker build -t $CI_REGISTRY_IMAGE:$APP_VERSION .
    - docker tag $CI_REGISTRY_IMAGE:$APP_VERSION $CI_REGISTRY_IMAGE:
       latest
    - docker push $CI_REGISTRY_IMAGE:$APP_VERSION
    - docker push $CI_REGISTRY_IMAGE:latest
  only:
    - main
```

Deploy Stage:

```
deploy-staging:
  stage: deploy
  image: docker:latest
  services:
    - docker:dind
  variables:
    DOCKER_TLS_CERTDIR: ""
  script:
    - echo "$CI_JOB_TOKEN" | docker login -u "$CI_REGISTRY_USER"
        --password-stdin $CI_REGISTRY
    - docker pull $CI_REGISTRY_IMAGE:$APP_VERSION
    - docker stop tp1-staging || true
```

```
    - docker rm tp1-staging || true
    - docker run -d --name tp1-staging -p 8000:3000
        $CI_REGISTRY_IMAGE:$APP_VERSION
  environment:
    name: staging
    url: http://staging.local:8000
  only:
    - main
```

6.3 Pipeline Execution — Success

After pushing the updated `.gitlab-ci.yml`, the pipeline #**2340412254** ran successfully across all stages: build, test, security, package, and deploy.

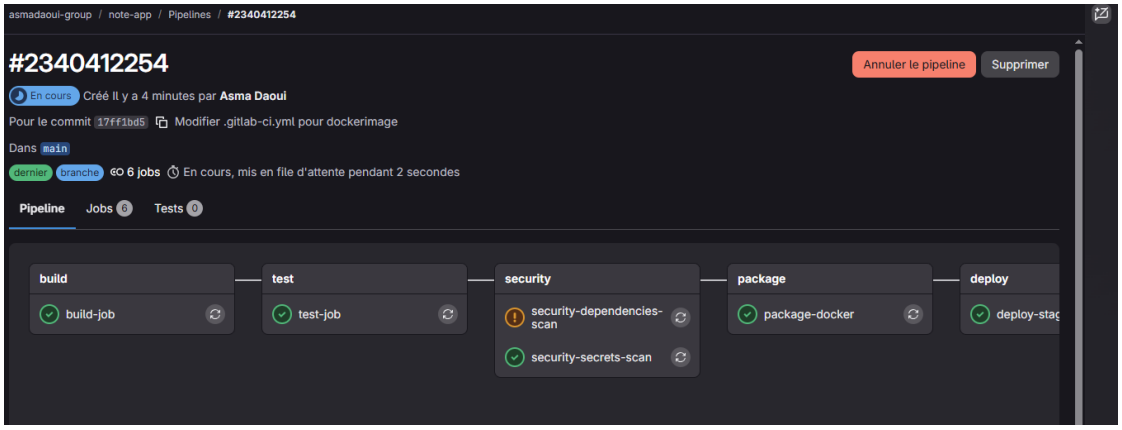

Figure 12: GitLab CI/CD pipeline #2340412254 — all stages completed successfully

6.4 Docker Image in GitLab Registry

After the pipeline completed, the GitLab Container Registry now shows the **asmadaoui-group/note-app** image with 2 tags, published successfully.

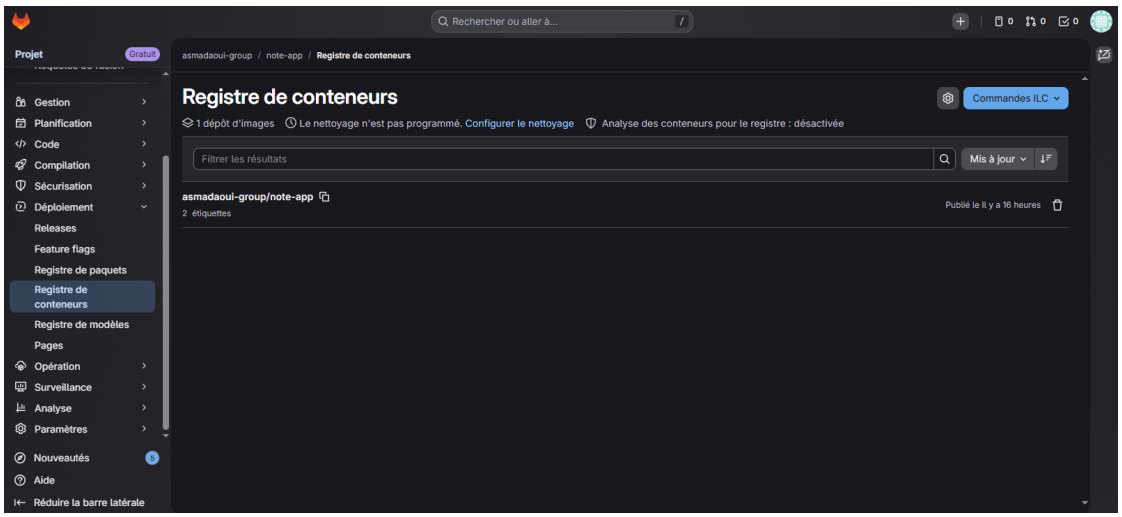

Figure 13: GitLab Container Registry — Docker image successfully pushed with 2 tags

7 Deploying the Application on EC2

7.1 Step 1 — Enable SSH and Connect to EC2

In the AWS Console, verify that the EC2 instance has SSH enabled. The security group must allow inbound TCP on port 22.

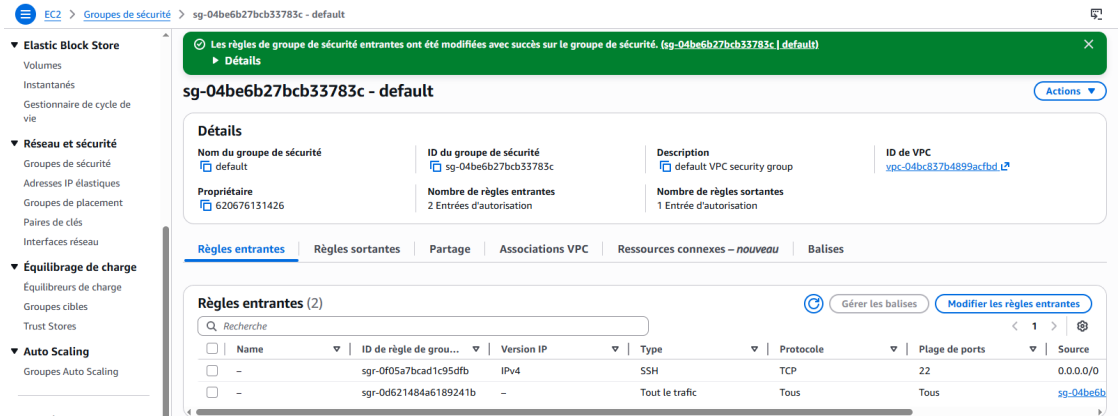

Figure 14: EC2 Security Group — SSH (port 22) inbound rule active

Copy the key pair from Windows to the Ubuntu WSL environment and connect:

```
cp /mnt/c/Users/HP/Downloads/terraformkeyinstance.pem ~/
chmod 400 ~/terraformkeyinstance.pem
ssh -i ~/terraformkeyinstance.pem ec2-user@100.25.217.89
```

7.2 Step 2 — Create a GitLab Personal Access Token

To authenticate Docker on the EC2 instance against the GitLab Container Registry, a Personal Access Token (PAT) is required (especially if 2FA is enabled). Navigate to **GitLab → User Settings → Personal Access Tokens** and create a token named `ec2-docker` with `read_registry` and `write_registry` scopes.

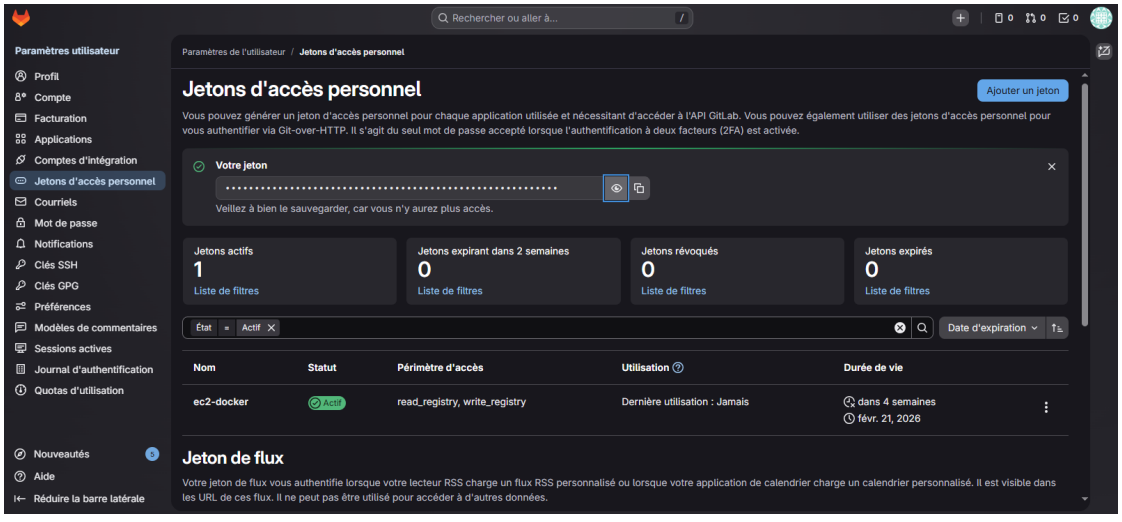

Figure 15: GitLab Personal Access Token — `ec2-docker` token created with registry permissions

Important: Save the token immediately after creation. GitLab will not show it again.

7.3 Step 3 — Install Docker on EC2

```
sudo yum update -y
sudo yum install docker -y
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker ec2-user
# Log out and reconnect via SSH after this step
```

7.4 Step 4 — Start Docker and Verify

After reconnecting, start and enable the Docker service, then check its status:

```
sudo systemctl start docker
sudo systemctl enable docker
sudo systemctl status docker
# Expected: Active: active (running)
```

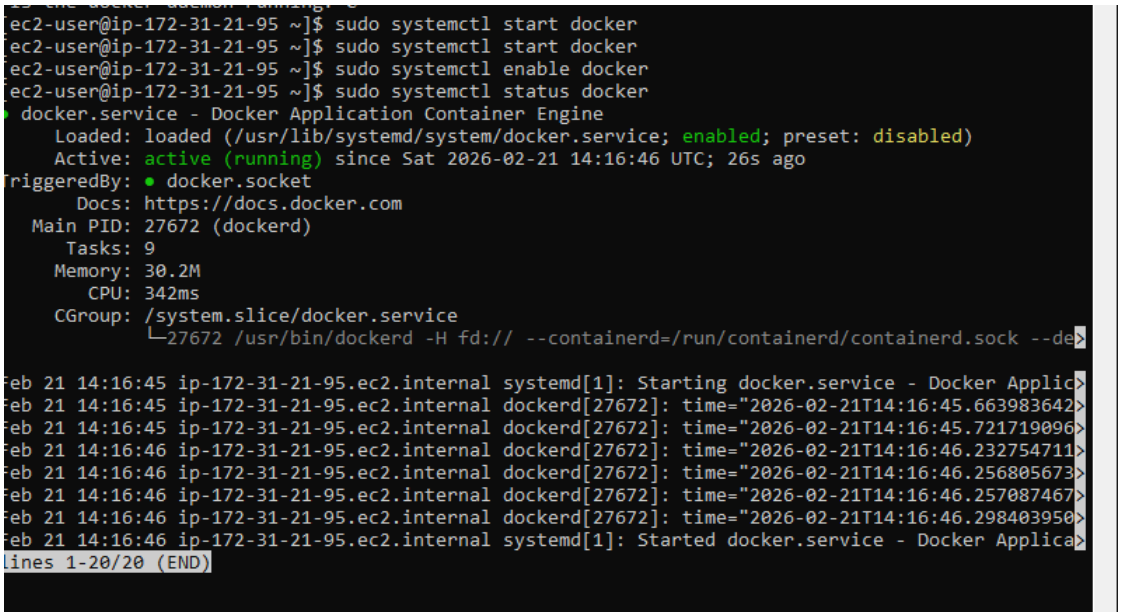

Figure 16: Docker service active and running on the EC2 instance

7.5 Step 5 — Pull and Run the Docker Image

Login to the GitLab Container Registry using the PAT, then pull and run the application:

```
docker login registry.gitlab.com
# Username: your GitLab username
# Password: your Personal Access Token

docker pull registry.gitlab.com/asmadaoui-group/note-app
```

```
docker run -d -p 80:3000 --name note-app \
  registry.gitlab.com/asmadaoui-group/note-app:latest
```

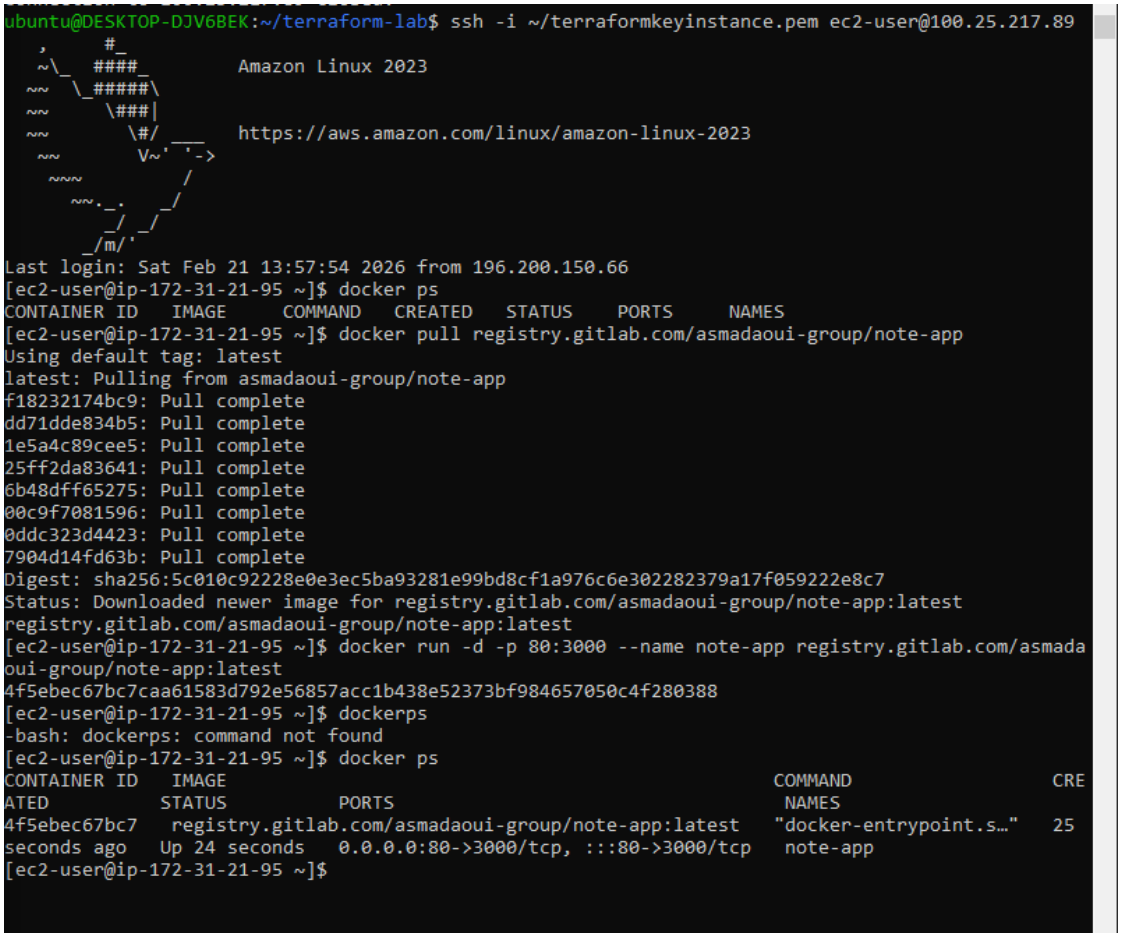

Figure 17: SSH into EC2 — Docker image pulled from GitLab and container running successfully

7.6 Step 6 — Open Port 8080 in the Security Group

To access the application from a browser, port **8080** must be added as an inbound rule in the EC2 security group. Navigate to **EC2** → **Security Groups** → **Inbound Rules** → **Edit**.

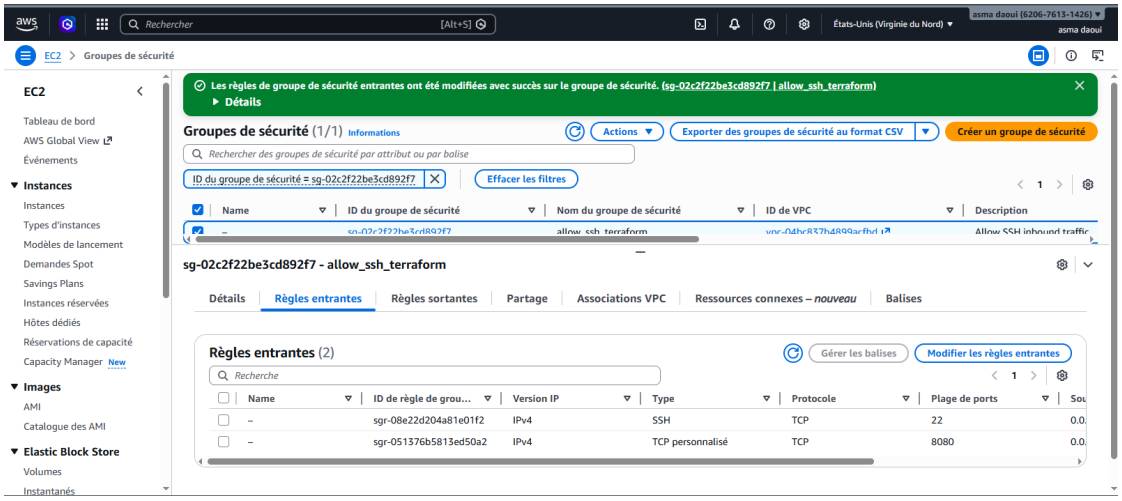

Figure 18: EC2 Security Group `allow_ssh_terraform` — inbound rules: SSH (22) and custom TCP (8080)

Result: The security group now allows both SSH access (port 22) and HTTP access to the application (port 8080) from anywhere (`0.0.0.0/0`).

7.7 Step 7 — Application Live in the Browser

After opening port 8080, the **My Notes** web application is accessible at `http://100.25.217.89:8080` directly from the browser.

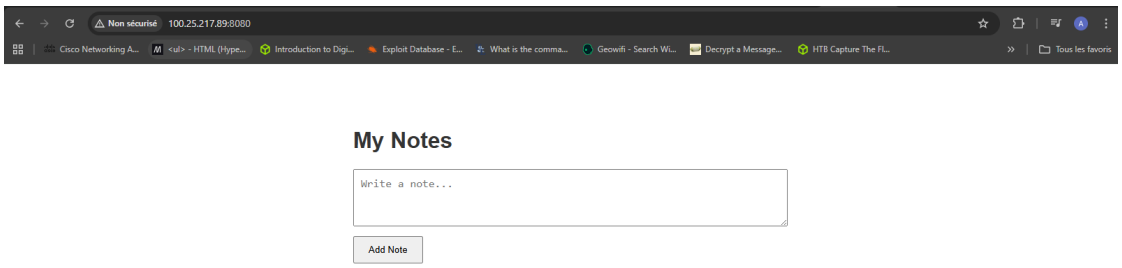

Figure 19: The Note App running live in the browser at `http://100.25.217.89:8080`

8 Summary

Resource / Step	Tool	Description
IAM User	AWS Console	Created with AdministratorAccess + S3FullAccess
Access Key	AWS IAM	CLI access key generated for Terraform
Key Pair	AWS EC2	SSH key pair `terraformkeyinstance`
S3 Bucket	Terraform	Object storage with Dev tag
Security Group	Terraform	SSH (22) + HTTP (8080) inbound rules
EC2 Instance	Terraform	`t3.micro`, Amazon Linux 2023
Docker Image	GitLab CI/CD	Built and pushed to GitLab Container Registry
App Deployment	Docker (EC2)	Container running on port 8080

Key Takeaways:

- Terraform enables reproducible, version-controlled infrastructure provisioning.
- IAM users with properly scoped policies are essential for secure CLI access.
- GitLab CI/CD automates Docker image build and push on every commit to `main`.
- Docker on EC2 allows seamless container-based deployment without orchestration overhead.
- Security groups must expose the correct ports (22 for SSH, 8080 for the app) for proper access.
- Personal Access Tokens are required for Docker registry login when 2FA is enabled on GitLab.